

LAPORAN PRAKTIKUM
STRUKTUR DATA
SINGLE LINKED LIST

Disusun Oleh :

Nama : Nabila Khairunnisa
Nim : 2511531003
Dosen Pengampu : Dr. Wahyudi, S.T, M.T
Asisten Praktikum : M. Galid Avero



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji Syukur atas kehadiran Allah SWT yang telah melimpahkan Rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum struktur data dengan judul “Single Linked List” dengan baik dan tepat waktu. Dalam menyelesaikan laporan ini saya banyak mendapat arahan dan bimbingan, oleh karena itu saya ingin mengucapkan terimakasih kepada

1. Bapak Dr. Wahyudi, S.T, M.T selaku dosen pengampu
2. Uda M. Galid Averro selaku asisten labor
3. Orang tua yang senantiasa mendoakan
4. Teman teman yang selalu memotivasi

Penulis menyadari bahwa laporan ini jauh dari kata sempurna. Oleh sebab itu, penulis sangat membuka diri apabila ada yang ingin memberikan kritikan dan saran yang sifatnya membantu, penulis akan sangat senang menerima. Tujuannya agar untuk kedepannya bisa menyempurnakan laporan.

Padang, 4 Mei 2026

Nabila Khairunnisa

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN.....	2
2.1 Deskripsi Praktikum.....	2
2.2 Langkah Langkah Praktikum.....	2
BAB III KESIMPULAN.....	8
3.1 Kesimpulan	8
3.2 Saran	8
DAFTAR PUSTAKA	9
LAPORAN TUGAS PEKAN 4	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Linked list adalah struktur data linier berbentuk rantai simpul di mana setiap simpul menyimpan 2 item, yaitu nilai data dan pointer ke simpul elemen berikutnya. Berbeda dengan array, elemen linked list tidak ditempatkan dalam alamat memori yang berdekatan melainkan elemen ditautkan menggunakan pointer.

Simpul pertama dari linked list disebut sebagai head atau simpul kepala. Apabila linked list berisi elemen kosong, maka nilai pointer dari head menunjuk ke NULL. Begitu juga untuk pointer berikutnya dari simpul terakhir atau simpul ekor akan menunjuk ke NULL.

Ukuran elemen dari linked list dapat bertambah secara dinamis dan mudah untuk menyisipkan dan menghapus elemen karena tidak seperti array, kita hanya perlu mengubah pointer elemen sebelumnya dan elemen berikutnya untuk menyisipkan atau menghapus elemen. [1]

Single linked list adalah jenis linked list paling sederhana, di mana setiap node hanya memiliki satu pointer yang menunjuk ke node berikutnya. Kelebihan dari single linked list yaitu struktur sederhana, penggunaan memori lebih hemat dan mudah diimplementasikan. Kekurangannya yaitu tidak bisa mundur ke node sebelumnya, proses pencarian harus dimulai dari awal dan akses data lebih lambat dibanding array.

Operasi dasar pada linked list seperti traversal untuk proses mengakses setiap node secara berurutan, insertion untuk penambahan data, deletion untuk menghapus data dan searching untuk pencarian. [2]

1.2 Tujuan Praktikum

1. Mampu memahami konsep dasar Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node).
2. Mampu menerapkan konsep FIFO
3. Mampu memahami cara kerja node pada linked list
4. Memahami penggunaan operasi dasar pada Linked List

1.3 Manfaat Praktikum

1. Mahasiswa mampu memahami penerapan Single Linked List.
2. Mahasiswa mampu memahami operasi dasar pada Linked List.
3. Mahasiswa mampu menerapkan prinsip FIFO
4. Mahasiswa mampu memahami cara kerja node pada Linked List.

BAB II

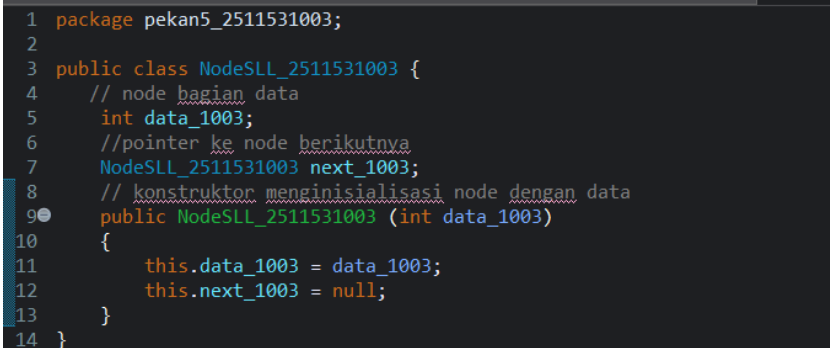
PEMBAHASAN

2.1 Deskripsi Praktikum

Praktikum Struktur data pada pekan ini adalah membuat Single Linked List yang diimplementasikan dengan menggunakan class `NodeSLL_2511531003` sebagai class untuk membuat node pada Single Linked List. Setiap node memiliki data dan pointer next yang digunakan untuk menghubungkan node satu dengan node berikutnya. Selanjutnya dibuat class utama untuk menjalankan operasi dasar pada Single Linked List. Operasi yang dilakukan yaitu menambahkan data, menghapus node pertama, menghapus node terakhir, menghapus node pada posisi tertentu serta menampilkan isi Single Linked List.

2.2 Langkah Langkah Praktikum

- 1) Buatlah package terlebih dahulu dengan mengklik kanan di folder `PrakSD2026_C_2511531003/src`, pilih `new` dan klik `package`. Setelah itu beri nama pada package tanpa huruf kapital, karakter khusus serta tanpa “space”. lalu “finish”.
- 2) Klik kanan pada package `pekan5_2511531003` yang sudah dibuat sebelumnya, pilih “New”, lalu pilih class. Buat nama “`NodeSLL_2511531003`” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini digunakan sebagai rancangan atau cetakan untuk membuat node pada Single Linked List.
- 3) Tuliskan program seperti berikut.

A screenshot of a code editor showing the implementation of the NodeSLL_2511531003 class. The code is written in Java and includes package declarations, class definition, and a constructor. The code is as follows:

```
1 package pekan5_2511531003;
2
3 public class NodeSLL_2511531003 {
4     // node bagian data
5     int data_1003;
6     //pointer ke node berikutnya
7     NodeSLL_2511531003 next_1003;
8     // konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511531003 (int data_1003)
10    {
11        this.data_1003 = data_1003;
12        this.next_1003 = null;
13    }
14 }
```

Gambar 2.2.1 kode program

- *int data_1003* digunakan untuk menyimpan data utama pada node dengan tipe data int.
 - *NodeSLL_2511531003 next_1003* berfungsi sebagai pointer atau penunjuk ke node berikutnya.
 - *public NodeSLL_2511531003 (int data_1003)* adalah bagian konstruktor. Konstruktor akan dijalankan secara otomatis ketika objek node dibuat.
 - *This.data_1003 = data_1003* berarti data dikirim melalui parameter akan disimpan ke dalam variabel milik class.
 - *this.next_1003 = null* berarti saat node pertama kali dibuat, node tersebut belum menunjuk ke node lain sehingga diatur menjadi null.
- 4) Buatlah class kedua dengan nama “PencarianSLL_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu centang tanda “public static void main (string[] args)” lalu “finish”. Class ini digunakan untuk membentuk data Single Linked List, menelusuri atau menampilkan semua data, dan mencari data tertentu dalam Linked List.
- 5) Tuliskan program seperti berikut.

```

1 package pekan5_2511531003;
2
3 public class PencarianSLL {
4     static boolean searchKey (NodeSLL_2511531003 head, int key_1003) {
5         NodeSLL_2511531003 curr_1003 = head;
6         while (curr_1003 != null) {
7             if (curr_1003.data_1003 == key_1003)
8                 return true;
9             curr_1003 = curr_1003.next_1003; }
10        return false; }
11    public static void traversal (NodeSLL_2511531003 head) {
12        //mulai dari head
13        NodeSLL_2511531003 curr_1003 = head;
14        //telusuri sampai pointer null
15        while (curr_1003 != null) {
16            System.out.print(" " + curr_1003.data_1003);
17            curr_1003 = curr_1003.next_1003; }
18        System.out.println (); }
19    public static void main(String[] args) {
20        NodeSLL_2511531003 head = new NodeSLL_2511531003 (14);
21        head.next_1003 = new NodeSLL_2511531003 (21);
22        head.next_1003.next_1003 = new NodeSLL_2511531003 (13);
23        head.next_1003.next_1003.next_1003 = new NodeSLL_2511531003 (30);
24        head.next_1003.next_1003.next_1003.next_1003 = new NodeSLL_2511531003 (10);
25        System.out.print("Penelusuran SLL : ");
26        traversal (head);
27        //data yang akan dicari
28        int key = 30;
29        System.out.print("cari data " +key+ " = ");
30        if (searchKey(head, key))
31            System.out.println("ketemu");
32        else
33            System.out.println("tidak ada");
34    }
35 }
36

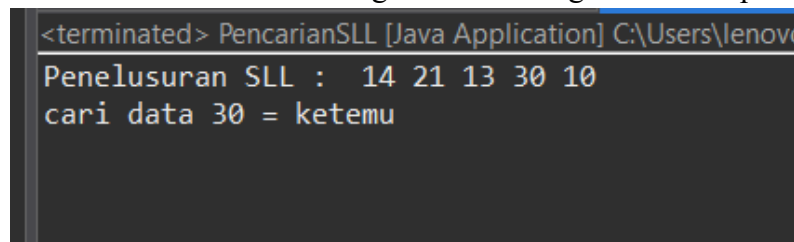
```

Gambar 2.2.2 Kode Program

- *searchKey* berfungsi untuk mencari apakah suatu data ada di dalam Linked List atau tidak.
- *NodeSLL_2511531003 curr_1003 = head* artinya pencarian dimulai dari head yaitu node pertama.

- *while (curr_1003 != null)* artinya program melakukan perulangan selama node saat ini tidak bernilai null.
- Method 7 dan 8 artinya jika data pada node saat ini sama dengan data yang dicari, maka program akan mengembalikan nilai true.
- Method 9 dan 10 artinya jika belum ketemu, pointer akan berpindah ke node berikutnya dan akan mengembalikan nilai false.
- Method *traversal* berfungsi untuk menampilkan seluruh data yang ada dalam Single Linked List dimulai dari head.
- Method 15 sampai 18 artinya selama node belum null, data akan ditampilkan ke layar. Setelah satu data ditampilkan, pointer berpindah ke node berikutnya sampai semua node selesai ditelusuri.
- Method *main* adalah bagian utama program yang akan dijalankan.
- Method 20 sampai 24 berfungsi untuk membuat beberapa node secara berurutan.
- *System.out.print* berfungsi adalah perintah untuk menampilkan data.
- *Int key = 30* artinya program mencari apakah data 30 ada di dalam Linked List.
- Melakukan pengecekan dengan if else

- 6) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut



```
<terminated> PencarianSLL [Java Application] C:\Users\lenovo
Penelusuran SLL : 14 21 13 30 10
cari data 30 = ketemu
```

Gambar 2.2.3 Output program

- 7) Class ketiga dengan nama “TambahSLL_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini berfungsi melakukan beberapa operasi penambahan node di depan, di belakang dan di posisi tertentu pada single linked list.

8) Tuliskan program seperti berikut.

```

1 package pekan5_2511531003;
2
3 public class TambahSLL_2511531003 {
4     public static NodeSLL_2511531003 insertAtFront (NodeSLL_2511531003 head, int value_1003) {
5         NodeSLL_2511531003 new_node = new NodeSLL_2511531003 (value_1003);
6         new_node.next_1003 = head;
7         return new_node;
8     }
9     //fungsi menambahkan node di akhir SLL
10    public static NodeSLL_2511531003 insertAtEnd (NodeSLL_2511531003 head, int value_1003) {
11        //buat sebuah node dengan sebuah nilai
12        NodeSLL_2511531003 new_node = new NodeSLL_2511531003 (value_1003);
13        //jika list kosong maka node menjadi head
14        if (head == null) {
15            return new_node;
16        }
17        //simpan head ke variabel sementara
18        NodeSLL_2511531003 last = head;
19        //keluarkan ke node akhir
20        while (last.next_1003 != null) {
21            last = last.next_1003;
22        }
23        //ubah pointer
24        last.next_1003 = new_node;
25        return head;
26    }
27    static NodeSLL_2511531003 GetNode (int data_1003) {
28        return new NodeSLL_2511531003 (data_1003);
29    }
30
31    static NodeSLL_2511531003 insertPos (NodeSLL_2511531003 headNode, int position_1003, int value_1003) {
32        NodeSLL_2511531003 head = headNode;
33        if (position_1003 < 1)
34            System.out.print("Invalid position");
35        if (position_1003 == 1) {
36            NodeSLL_2511531003 new_node = new NodeSLL_2511531003 (value_1003);
37            new_node.next_1003 = head;
38            return new_node;
39        }
40        else {
41            while (position_1003-- != 0) {
42                if (position_1003 == 1) {
43                    NodeSLL_2511531003 new_node = GetNode (value_1003);
44                    new_node.next_1003 = headNode.next_1003;
45                    headNode.next_1003 = new_node;
46                    break;
47                }
48                headNode = headNode.next_1003;
49            }
50            if (position_1003 != 1)
51                System.out.print("Posisi di luar jangkauan");
52            return head;
53        }
54        public static void printList (NodeSLL_2511531003 head) {
55            NodeSLL_2511531003 curr = head;
56            while (curr.next_1003 != null) {
57                System.out.print(curr.data_1003+"->");
58                curr = curr.next_1003;
59            }
60            if (curr.next_1003 == null) {
61                System.out.print(curr.data_1003);
62                System.out.println();
63            }
64        }
65
66        public static void main(String[] args) {
67            //buat linked list 2->3->5->6
68            NodeSLL_2511531003 head = new NodeSLL_2511531003(2);
69            head.next_1003 = new NodeSLL_2511531003 (3);
70            head.next_1003.next_1003 = new NodeSLL_2511531003 (5);
71            head.next_1003.next_1003.next_1003 = new NodeSLL_2511531003 (6);
72            //cetak list asli
73            System.out.print("Senarai berantai awal:");
74            printList(head);
75            //tambahkan node baru di depan
76            System.out.print("tambah 1 simpul di depan: ");
77            int data = 1;
78            head = insertAtFront (head, data);
79            //cetak update list
80            printList(head);
81            //tambahkan node baru dibelakang
82            System.out.print("tambah 1 simpul di belakang: ");
83            int data2 = 7;
84            head = insertAtEnd (head, data2);
85            //cetak update list
86            printList(head);
87            System.out.print("tambah 1 simpul ke data ke 4: ");
88            int data3 = 4;
89            int pos = 4;
90            head = insertPos(head,pos,data3);
91            //cetak update list
92            printList(head);
93        }
94    }
95 }

```

Gambar 2.2.4 Kode Program

- *insertAtFront* untuk menambahkan node di awal list.
- *insertAtEnd* untuk menambahkan node di akhir list.
- *GetNode* untuk membuat node baru.
- *insertPos* untuk menambahkan node pada posisi tertentu.
- *printList* untuk menampilkan isi linked list
- *main* untuk menjalankan program

- 9) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```
<terminated> TambahSSL_2511531003 [Java Application] C:\Users\lenovo\.p2\pool\plu
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data ke 4: 1-->2-->3-->4-->5-->6-->7
```

Gambar 2.2.5 Output Program

- 10) Class ke empat dengan nama “HapusSLL_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini berfungsi untuk melakukan operasi penghapusan dan pencetakan data pada linked list.

- 11) Tuliskan program seperti berikut

```
1 package pekan5_2511531003;
2
3 public class HapusSLL_2511531003 {
4     //fungsi untuk menghapus head
5     public static NodeSLL_2511531003 deleteHead (NodeSLL_2511531003 head) {
6         //jika SLL kosong
7         if (head == null)
8             return null;
9         //pindahkan head ke node berikutnya
10        head = head.next_1003;
11        //return head baru
12        return head;
13    }
14    //fungsi menghapus node terakhir SLL
15    public static NodeSLL_2511531003 removeLastNode (NodeSLL_2511531003 head_1003) {
16        //jika list kosong, return null
17        if (head_1003 == null) {
18            return null;
19        }
20        //jika list satu node, hapus node dan return null
21        if (head_1003.next_1003 == null) {
22            return null;
23        }
24        //temukan node terakhir ke dua
25        NodeSLL_2511531003 secondLast = head_1003;
26        while (secondLast.next_1003.next_1003 != null) {
27            secondLast = secondLast.next_1003;
28        }
29        //hapus node terakhir
30        secondLast.next_1003 = null;
31        return head_1003;
32    }
33    //fungsi menghapus node di posisi tertentu
34    public static NodeSLL_2511531003 deleteNode (NodeSLL_2511531003 head, int position_1003) {
35        NodeSLL_2511531003 temp = head;
36        NodeSLL_2511531003 prev = null;
37
38        //jika linked list null
39        if (temp == null)
40            return head;
41        //kasus 1 : head di hapus
42        if (position_1003 == 1) {
43            head = temp.next_1003;
44            return head;
45        }
46        //kasus 2: menghapus node di tengah
47        //telusuri ke node yang di hapus
48        for (int i = 1; temp != null && i < position_1003; i++) {
49            prev = temp;
50            temp = temp.next_1003;
51        }
52        //jika ditemukan, hapus node
53        if (temp != null) {
54            prev.next_1003 = temp.next_1003;
55        }
56        else {
57            System.out.println("Data tidak ada");
58            return head;
59        }
60        //fungsi mencetak SLL
61        public static void printlist (NodeSLL_2511531003 head) {
62            NodeSLL_2511531003 curr = head;
63            while (curr.next_1003 != null) {
64                System.out.print(curr.data_1003+"-->");
65                curr = curr.next_1003;
66            }
67            if (curr.next_1003==null) {
68                System.out.print(curr.data_1003);
69            }
70            System.out.println();
71        }
72    }
73 }
```

```

63 //kelas main
64 public static void main(String[] args) {
65     //buat SLL 1->2->3->4->5->6->null
66     NodeSLL_2511531003 head = new NodeSLL_2511531003 (1);
67     head.next_1003 = new NodeSLL_2511531003 (2);
68     head.next_1003.next_1003 = new NodeSLL_2511531003 (3);
69     head.next_1003.next_1003.next_1003 = new NodeSLL_2511531003 (4);
70     head.next_1003.next_1003.next_1003.next_1003 = new NodeSLL_2511531003 (5);
71     head.next_1003.next_1003.next_1003.next_1003.next_1003 = new NodeSLL_2511531003 (6);
72     //cetak list awal
73     System.out.println ("list awal: ");
74     printList(head);
75     //hapus head
76     head = deleteHead (head);
77     System.out.println ("list setelah head dihapus: ");
78     printList(head);
79     //hapus node terakhir
80     head = removeLastNode (head);
81     System.out.println ("list setelah simpul reakhir di hapus: ");
82     printList(head);
83     //Deleting node at position 2
84     int position = 2;
85     head = deleteNode (head, position);
86     //print list after deletion
87     System.out.println ("list setelah posisi 2 dihapus: ");
88     printList(head);
89 }
90 }

```

Gambar 2.2.6 Kode Program

- *deleteHead* untuk menghapus node pertama pada linked list.
- *If (head == null) return null* artinya jika linked list masih kosong, maka program akan mengembalikan nilai null.
- *removeLastNode* berfungsi untuk menghapus node terakhir pada linked list.
- *secondLast* berfungsi untuk mencari node kedua terakhir.
- *deleteNode* berfungsi untuk menghapus node pada posisi tertentu.
- *printList* berfungsi untuk mencetak isi linked list.
- Variabel *curr* digunakan untuk menelusuri linked list dari awal sampai akhir.

12) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```

<terminated> HapusSLL_2511531003 [Java Application] C:\Users\len
list awal:
1-->2-->3-->4-->5-->6
list setelah head dihapus:
2-->3-->4-->5-->6
list setelah simpul reakhir di hapus:
2-->3-->4-->5
list setelah posisi 2 dihapus:
2-->4-->5

```

Gambar 2.2.7 Output Program

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Single Linked List bekerja secara dinamis yang terdiri dari beberapa node yang saling terhubung melalui pointer next. Setiap node menyimpan data dan Alamat node berikutnya, sehingga data dapat disusun secara berurutan tanpa harus menggunakan ukuran tetap. Pada praktikum ini, program berhasil menerapkan beberapa operasi dasar pada Single Linked List seperti menampilkan isi list, menghapus node pertama, menghapus node terakhir serta menghapus node pada posisi tertentu.

3.2 Saran

Untuk menambah pengembangan praktikum di masa mendatang disarankan untuk pemahaman terhadap konsep head, next dan perpindahan pointer perlu diperhatikan karena kesalahan kecil pada bagian tersebut bisa menyebabkan error.

DAFTAR PUSTAKA

- [1] Trivusi, "Struktur Data Linked List: Pengertian, Karakteristik, dan Jenis-jenisnya," trivusi.web.id, 16 September 2022. [Online]. Available: https://www.trivusi.web.id/2022/07/struktur-data-linked-list.html#google_vignette. [Accessed 6 Mei 2026].

- [2] Dianaape, "Linked List: Single, Double, dan Circular Linked Lis," Telkom University Jakarta, 13 Februari 2026. [Online]. Available: <https://jakarta.telkomuniversity.ac.id/linked-list-single-double-dan-circular-linked-list/>. [Accessed 6 Mei 2026].

LAPORAN TUGAS PRAKTIKUM PEKAN 5

1. KODE PROGRAM

- **//Class 1 : Pasien_2511531003**

```
package pekan5_2511531003;
```

```
public class Pasien_2511531003 {
    String namaPasien_1003;
    String penyakit_1003;
    int nomorAntrian_1003;
    Pasien_2511531003 next_1003;

    // Constructor
    public Pasien_2511531003(String namaPasien_1003, String penyakit_1003, int
nomorAntrian_1003) {
        this.namaPasien_1003 = namaPasien_1003;
        this.penakit_1003 = penyakit_1003;
        this.nomorAntrian_1003 = nomorAntrian_1003;
        this.next_1003 = null;
    }

    // Getter
    public String getNamaPasien_1003() {
        return namaPasien_1003;
    }

    public String getPenyakit_1003() {
        return penyakit_1003;
    }

    public int getNomorAntrian_1003() {
        return nomorAntrian_1003;
    }

    public Pasien_2511531003 getNext_1003() {
        return next_1003;
    }
}
```

```

// Setter
public void setNamaPasien_1003(String namaPasien_1003) {
    this.namaPasien_1003 = namaPasien_1003;
}

public void setPenyakit_1003(String penyakit_1003) {
    this.penyakit_1003 = penyakit_1003;
}

public void setNomorAntrian_1003(int nomorAntrian_1003) {
    this.nomorAntrian_1003 = nomorAntrian_1003;
}

public void setNext_1003(Pasien_2511531003 next_1003) {
    this.next_1003 = next_1003;
}
}

```

- **//Class 2 : RumahSakit_2511531003**

```

package pekan5_2511531003;
import java.util.Scanner;
public class RumahSakit_2511531003 {
    Pasien_2511531003 head_1003;
    int counter_1003;

    public RumahSakit_2511531003() {
        head_1003 = null;
        counter_1003 = 0;
    }

    // 1. Daftarkan Pasien (Insert)
    public void daftarkanPasien_1003(String namaPasien_1003, String
    penyakit_1003) {
        counter_1003++;

        Pasien_2511531003 pasienBaru_1003 =

```

```

        new Pasien_2511531003(namaPasien_1003, penyakit_1003,
counter_1003);

```

```

        if (head_1003 == null) {
            head_1003 = pasienBaru_1003;
        } else {
            Pasien_2511531003 temp_1003 = head_1003;

            while (temp_1003.getNext_1003() != null) {
                temp_1003 = temp_1003.getNext_1003();
            }

            temp_1003.setNext_1003(pasienBaru_1003);
        }

```

```

        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " +
counter_1003);
    }

```

// 2. Panggil Pasien (Delete head)

```

public void panggilPasien_1003() {
    if (head_1003 == null) {
        System.out.println("Antrian masih kosong.");
    } else {
        System.out.println("Pasien yang dipanggil:");
        System.out.println("Nomor Antrian : " +
head_1003.getNomorAntrian_1003());
        System.out.println("Nama Pasien : " +
head_1003.getNamaPasien_1003());
        System.out.println("Keluhan : " + head_1003.getPenyakit_1003());

        head_1003 = head_1003.getNext_1003();
    }
}

```

// 3. Tampilkan Antrian (Display)

```

public void tampilkanAntrian_1003() {
    if (head_1003 == null) {
        System.out.println("Antrian masih kosong.");
    } else {

```

```

        Pasien_2511531003 temp_1003 = head_1003;

        System.out.println("Daftar Antrian Pasien:");
        while (temp_1003 != null) {
            System.out.println("-----");
            System.out.println("Nomor      Antrian      :      "      +
temp_1003.getNomorAntrian_1003());
            System.out.println("Nama      Pasien      :      "      +
temp_1003.getNamaPasien_1003());
            System.out.println("Keluhan      : " + temp_1003.getPenyakit_1003());

            temp_1003 = temp_1003.getNext_1003();
        }
    }
}

```

// 4. Cari Pasien (Search)

```

public void cariPasien_1003(String namaCari_1003) {
    if (head_1003 == null) {
        System.out.println("Antrian masih kosong.");
        return;
    }

    Pasien_2511531003 temp_1003 = head_1003;
    boolean ditemukan_1003 = false;

    while (temp_1003 != null) {
        if
(temp_1003.getNamaPasien_1003().equalsIgnoreCase(namaCari_1003)) {
            System.out.println("Pasien ditemukan:");
            System.out.println("Nomor      Antrian      :      "      +
temp_1003.getNomorAntrian_1003());
            System.out.println("Nama      Pasien      :      "      +
temp_1003.getNamaPasien_1003());
            System.out.println("Keluhan      : " + temp_1003.getPenyakit_1003());
            ditemukan_1003 = true;
            break;
        }

        temp_1003 = temp_1003.getNext_1003();
    }
}

```



```

    }

    if (!ditemukan_1003) {
        System.out.println("Pasien tidak ditemukan.");
    }
}

// 5. Cek Status Antrian
public void cekStatusAntrian_1003() {
    if (head_1003 == null) {
        System.out.println("Antrian masih kosong.");
        return;
    }

    int jumlah_1003 = 0;
    Pasien_2511531003 temp_1003 = head_1003;

    while (temp_1003 != null) {
        jumlah_1003++;
        temp_1003 = temp_1003.getNext_1003();
    }

    System.out.println("Jumlah total pasien dalam antrian: " + jumlah_1003);
    System.out.println("Pasien terdepan:");
    System.out.println("Nomor      Antrian      :      "      +
head_1003.getNomorAntrian_1003());
    System.out.println("Nama Pasien  : " + head_1003.getNamaPasien_1003());
    System.out.println("Keluhan    : " + head_1003.getPenyakit_1003());
    }

// Main Program
public static void main(String[] args) {
    Scanner input_1003 = new Scanner(System.in);
    RumahSakit_2511531003 rs_1003 = new RumahSakit_2511531003();

    int pilihan_1003;

    do {
        System.out.println("\n===== Antrian Rumah Sakit NIM: 2511531003
=====");

```

```

System.out.println("1. Daftarkan Pasien (Insert)");
System.out.println("2. Panggil Pasien (Delete Head)");
System.out.println("3. Tampilkan Antrian (Display)");
System.out.println("4. Cari Pasien (Search)");
System.out.println("5. Cek Status Antrian");
System.out.println("6. Keluar");
System.out.print("Pilihan: ");
pilihan_1003 = input_1003.nextInt();
input_1003.nextLine();

switch (pilihan_1003) {
    case 1:
        System.out.print("Masukkan Nama Pasien : ");
        String namaPasien_1003 = input_1003.nextLine();

        System.out.print("Masukkan Keluhan : ");
        String penyakit_1003 = input_1003.nextLine();

        rs_1003.daftarkanPasien_1003(namaPasien_1003, penyakit_1003);
        break;

    case 2:
        rs_1003.panggilPasien_1003();
        break;

    case 3:
        rs_1003.tampilkanAntrian_1003();
        break;

    case 4:
        System.out.print("Masukkan nama pasien yang dicari: ");
        String namaCari_1003 = input_1003.nextLine();

        rs_1003.cariPasien_1003(namaCari_1003);
        break;

    case 5:
        rs_1003.cekStatusAntrian_1003();
        break;
}

```

```

        case 6:
            System.out.println("Program selesai. Terima kasih.");
            break;

        default:
            System.out.println("Pilihan tidak valid.");
            break;
    }

    } while (pilihan_1003 != 6);

    input_1003.close();
}
}

```

2. SCREENSHOT CODE PROGRAM

- Class 1 : Pasien_2511531003

```

1 package pekan5_2511531003;
2
3 public class Pasien_2511531003 {
4     String namaPasien_1003;
5     String penyakit_1003;
6     int nomorAntrian_1003;
7     Pasien_2511531003 next_1003;
8
9     // Constructor
10    public Pasien_2511531003(String namaPasien_1003, String penyakit_1003, int nomorAntrian_1003) {
11        this.namaPasien_1003 = namaPasien_1003;
12        this.penakit_1003 = penyakit_1003;
13        this.nomorAntrian_1003 = nomorAntrian_1003;
14        this.next_1003 = null;
15    }
16
17    // Getter
18    public String getNamaPasien_1003() {
19        return namaPasien_1003;
20    }
21
22    public String getPenyakit_1003() {
23        return penyakit_1003;
24    }
25
26    public int getNomorAntrian_1003() {
27        return nomorAntrian_1003;
28    }
29
30    public Pasien_2511531003 getNext_1003() {
31        return next_1003;
32    }
33
34    // Setter
35    public void setNamaPasien_1003(String namaPasien_1003) {
36        this.namaPasien_1003 = namaPasien_1003;
37    }
38
39    public void setPenyakit_1003(String penyakit_1003) {
40        this.penakit_1003 = penyakit_1003;
41    }
42
43    public void setNomorAntrian_1003(int nomorAntrian_1003) {
44        this.nomorAntrian_1003 = nomorAntrian_1003;
45    }
46
47    public void setNext_1003(Pasien_2511531003 next_1003) {
48        this.next_1003 = next_1003;
49    }
50 }

```

- **Class 2 : RumahSakit_2511531003**

```

1 package pekan5_2511531003;
2 import java.util.Scanner;
3 public class RumahSakit_2511531003 {
4     Pasien_2511531003 head_1003;
5     int counter_1003;
6
7     public RumahSakit_2511531003() {
8         head_1003 = null;
9         counter_1003 = 0;
10    }
11
12    // 1. Daftarkan Pasien (Insert)
13    public void daftarkanPasien_1003(String namaPasien_1003, String penyakit_1003) {
14        counter_1003++;
15
16        Pasien_2511531003 pasienBaru_1003 =
17            new Pasien_2511531003(namaPasien_1003, penyakit_1003, counter_1003);
18
19        if (head_1003 == null) {
20            head_1003 = pasienBaru_1003;
21        } else {
22            Pasien_2511531003 temp_1003 = head_1003;
23
24            while (temp_1003.getNext_1003() != null) {
25                temp_1003 = temp_1003.getNext_1003();
26            }
27
28            temp_1003.setNext_1003(pasienBaru_1003);
29        }
30
31        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counter_1003);
32    }
33
34    // 2. Panggil Pasien (Delete head)
35    public void panggilPasien_1003() {
36        if (head_1003 == null) {
37            System.out.println("Antrian masih kosong.");
38        } else {
39            System.out.println("Pasien yang dipanggil:");
40            System.out.println("Nomor Antrian : " + head_1003.getNomorAntrian_1003());
41            System.out.println("Nama Pasien : " + head_1003.getNamaPasien_1003());
42            System.out.println("Keluhan : " + head_1003.getPenyakit_1003());
43
44            head_1003 = head_1003.getNext_1003();
45        }
46    }
47
48    // 3. Tampilkan Antrian (Display)
49    public void tampilkanAntrian_1003() {
50        if (head_1003 == null) {
51            System.out.println("Antrian masih kosong.");
52        } else {
53            Pasien_2511531003 temp_1003 = head_1003;
54
55            System.out.println("Daftar Antrian Pasien:");
56            while (temp_1003 != null) {
57                System.out.println("-----");
58                System.out.println("Nomor Antrian : " + temp_1003.getNomorAntrian_1003());
59                System.out.println("Nama Pasien : " + temp_1003.getNamaPasien_1003());
60                System.out.println("Keluhan : " + temp_1003.getPenyakit_1003());
61
62                temp_1003 = temp_1003.getNext_1003();
63            }
64        }
65    }
66
67    // 4. Cari Pasien (Search)
68    public void cariPasien_1003(String namaCari_1003) {
69        if (head_1003 == null) {
70            System.out.println("Antrian masih kosong.");
71            return;
72        }
73
74        Pasien_2511531003 temp_1003 = head_1003;
75        boolean ditemukan_1003 = false;
76
77        while (temp_1003 != null) {
78            if (temp_1003.getNamaPasien_1003().equalsIgnoreCase(namaCari_1003)) {
79                System.out.println("Pasien ditemukan:");
80                System.out.println("Nomor Antrian : " + temp_1003.getNomorAntrian_1003());
81                System.out.println("Nama Pasien : " + temp_1003.getNamaPasien_1003());
82                System.out.println("Keluhan : " + temp_1003.getPenyakit_1003());
83                ditemukan_1003 = true;
84                break;
85            }
86
87            temp_1003 = temp_1003.getNext_1003();
88        }
89
90        if (!ditemukan_1003) {
91            System.out.println("Pasien tidak ditemukan.");
92        }
93    }
94
95    // 5. Cek Status Antrian
96    public void cekStatusAntrian_1003() {
97        if (head_1003 == null) {
98            System.out.println("Antrian masih kosong.");
99            return;
100        }
101    }

```

```

102     int jumlah_1003 = 0;
103     Pasien_2511531003 temp_1003 = head_1003;
104
105     while (temp_1003 != null) {
106         jumlah_1003++;
107         temp_1003 = temp_1003.getNext_1003();
108     }
109
110     System.out.println("Jumlah total pasien dalam antrian: " + jumlah_1003);
111     System.out.println("Pasien terdepan:");
112     System.out.println("Nomor Antrian : " + head_1003.getNomorAntrian_1003());
113     System.out.println("Nama Pasien : " + head_1003.getNamaPasien_1003());
114     System.out.println("Keluhan : " + head_1003.getPenyakit_1003());
115 }
116
117 // Main Program
118 public static void main(String[] args) {
119     Scanner input_1003 = new Scanner(System.in);
120     RumahSakit_2511531003 rs_1003 = new RumahSakit_2511531003();
121
122     int pilihan_1003;
123
124     do {
125         System.out.println("\n===== Antrian Rumah Sakit NIM: 2511531003 =====");
126         System.out.println("1. Daftarkan Pasien (Insert)");
127         System.out.println("2. Panggil Pasien (Delete Head)");
128         System.out.println("3. Tampilkan Antrian (Display)");
129         System.out.println("4. Cari Pasien (Search)");
130         System.out.println("5. Cek Status Antrian");
131         System.out.println("6. Keluar");
132         System.out.print("Pilihan: ");
133         pilihan_1003 = input_1003.nextInt();
134         input_1003.nextLine();
135
136         switch (pilihan_1003) {
137             case 1:
138                 System.out.print("Masukkan Nama Pasien : ");
139                 String namaPasien_1003 = input_1003.nextLine();
140
141                 System.out.print("Masukkan Keluhan : ");
142                 String penyakit_1003 = input_1003.nextLine();
143
144                 rs_1003.daftarkanPasien_1003(namaPasien_1003, penyakit_1003);
145                 break;
146
147             case 2:
148                 rs_1003.panggilPasien_1003();
149                 break;
150
151             case 3:
152                 rs_1003.tampilkanAntrian_1003();
153                 break;
154
155             case 4:
156                 System.out.print("Masukkan nama pasien yang dicari: ");
157                 String namaCari_1003 = input_1003.nextLine();
158
159                 rs_1003.cariPasien_1003(namaCari_1003);
160                 break;
161
162             case 5:
163                 rs_1003.cekStatusAntrian_1003();
164                 break;
165
166             case 6:
167                 System.out.println("Program selesai. Terima kasih.");
168                 break;
169
170             default:
171                 System.out.println("Pilihan tidak valid.");
172                 break;
173
174         }
175     } while (pilihan_1003 != 6);
176
177     input_1003.close();
178 }
179

```

3. OUTPUT

```

RumahSakit_2511531003 [Java Application] C:\Users\lenovo\p2\pool\plugins\org.eclipse.justj.open
===== Antrian Rumah Sakit NIM: 2511531003 =====
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Budi Santoso
Masukkan Keluhan : Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

```

4. PENJELASAN TUGAS

Pada tugas pekan 4 ini, diperintahkan untuk membuat sebuah program dengan implementasi Single Linked List. Pada tugas kali ini adalah simulasi antrian rumah sakit. Program yang dibuat diwajibkan untuk mensimulasikan sistem pendaftaran antrian pasien, dimana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO). Program ini terdiri dari 2 class yaitu kelas **Pasien_2511531003** dan kelas **RumahSakit_2511531003**.

Pada kelas **Pasien_2511531003** merupakan kelas yang digunakan sebagai node dalam Single Linked List. Di dalam kelas ini terdapat atribut *namaPasien_1003*, *penyakit_1003*, *nomorAntrian_1003* dan *next_1003*. Selain itu, class ini memiliki constructor untuk menginisialisasi data pasien serta method getter dan setter untuk mengambil dan mengubah nilai setiap atribut.

Pada kelas **RumahSakit_2511531003** merupakan kelas driver yang digunakan untuk menjalankan program dan mengelola sistem antrian rumah sakit. Kelas ini berisi atribut *head_1003* sebagai penanda node pertama antrian, *counter_1003* sebagai penghitung nomor antrian pasien. Pada kelas ini terdapat beberapa method seperti *panggilPasien_1003()* untuk menambahkan pasien baru ke bagian akhir linked list, *tampilkanAntrian_1003()* untuk menampilkan seluruh daftar pasien, *cariPasien_1003()* untuk mencari pasien berdasarkan nama secara case-insensitive, dan *cekStatusAntrian_1003()* untuk menampilkan jumlah pasien serta informasi pasien terdepan.